
REINFORCEMENT LEARNING: SAMPLE-EFFICIENT BIPEDAL WALKER LEARNING WITH TD7

Ben Gilroy

Department of Computer Science
Durham University
xjbg74@durham.ac.uk

ABSTRACT

This paper proposes a solution to the Gymnasium bipedal walker task by implementing the TD7 algorithm, which extends TD3 by incorporating key improvements, including a state-action representation learning method (SALE), a prioritised experience replay and checkpoints. The TD7 model is evaluated on both easy and hardcore environments to test the model’s limits on challenging terrain. All training and experiments use a fixed seed of 42 for reproducibility. Results demonstrate that TD7 offers high sample efficiency while consistently reaching high rewards of 300 and above.

1 INTRODUCTION

In recent years, research has primarily focused on creating sample efficient solutions [1]. The Gymnasium bipedal walker task is a widely used benchmark for continuous control scenarios in reinforcement learning. In this task, the bipedal walker must learn how to traverse terrain efficiently while being sample efficient. The easy environment provides a flat, obstacle-free terrain that allows the agent to quickly learn how to walk. In contrast, the hardcore environment introduces uneven terrain and obstacles that require balance and precise foot placement.

In this paper, we propose the TD7 model [2] to solve the bipedal walker task due to its sample efficiency and stability. We introduce experiments aiming to improve the model in our environment. The rest of this paper is organized as follows. We first review related work in continuous-control reinforcement learning. Next, we discuss the TD7 algorithm and its implementation. We then describe our experimental setup and present quantitative results and analyses. We conclude with a discussion of limitations and directions for future research.

2 RELATED WORK & THEORY

One such model is the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm; an off-policy actor-critic method designed for continuous action spaces, introduced in 2018 [3]. The TD3 algorithm is an extension of the DDPG algorithm [4]. TD3 learns a deterministic policy alongside two Q-value critics, taking the minimum of their estimates to counteract value overestimation. It maintains target networks for both actor and critic and relies on a replay buffer, enabling both online and offline training. To further stabilize learning, TD3 updates the policy and target networks less frequently than the critics and adds small noise to the target action during critic updates, preventing the policy from exploiting overestimated values. Despite these improvements over DDPG, there remains room for enhanced sample efficiency and stability.

The Dreamwalker agent [5] leverages a mixture of modern off-policy and model-based techniques to boost sample efficiency on bipedal walker environments. It implements Truncated Quantile Critics (TQC) [6]. A distributional critic ensemble that clips overestimation bias, enhanced with D2RL’s dense MLP connections [7] to preserve state-action information at

every layer. An Emphasizing Recent Experience (ERE) [8] buffer further prioritizes fresh transitions, accelerating learning from the most relevant data. Crucially, Dreamwalker incorporates a Dreamer-style auto-regressive transformer [9] world model trained on past trajectories, enabling “imagination” rollouts that reduce reliance on real environment steps. Dreamwalker’s world model relies on a simplistic architecture and loss function that may inadequately capture environment dynamics or curb overestimation bias. Its transformer-based training is computationally intensive, especially with high episode counts, making it unfeasible for our bipedal walker environment. Due to the parameters being trained on the BipedalWalker V2 environment, it performed poorly on the rldurham walker.

TD7 [2] extends TD3 by integrating state-action learned embeddings (SALE) [10] and policy checkpoints. SALE offers a novel representation learning approach for low-level continuous states, jointly encoding state and action to capture environment dynamics, while checkpoints preserve the best-performing policies during training to prevent regressions. When applied to standard OpenAI Gym continuous-control benchmarks [11], TD7 achieves an average performance gain of 276.7% at 300k steps and 50.7% at 5M steps over TD3, demonstrating improved sample efficiency and robustness in both online and offline settings [2].

3 METHODOLOGY

The environment used in this project is the bipedal walker from the gymnasium reinforcement learning toolkit. The robot has two legs, each with a hip and knee joint, and a central hull. The continuous action space values are in the range of $[-1, 1]$ shown in the table Below. (referenced here ):

	Name	Min	Max
0	Hip 1 (Torque/velocity)	-1	+1
1	Knee 1 (Torque/velocity)	-1	+1
2	Hip 2 (Torque/velocity)	-1	+1
3	Knee 2 (Torque/velocity)	-1	+1

Table 1: The continuous action space is between -1 and $+1$

The environment has two versions, Normal and Hardcore, each with different terrain. The normal version is composed of flat and slightly uneven terrain which results in an easier environment to learn how to walk. On the other hand, the hardcore version includes ladders, stumps, and pitfalls, which add complexity and require a longer time for the agent to learn how to walk. The observation space contains hull kinematics, joint positions, joint velocities, ground contact flags, and LIDAR-based terrain sensing. It’s a common pitfall in the early stages of training to get stuck in a balanced position. By increasing the learning rate, you can prevent this issue. An optimal agent will quickly overcome this minima in reward and get better scores.

The starting state remains consistent each episode with the hull horizontal and both legs in the same position with a slight knee angle. Episode termination happens only when the hull gets in contact with the ground or if the walker reaches the end of the terrain length or runs out of time steps. This termination condition is much more common in the hardcore version due to the added obstacles, which require the agent to learn complex stability behaviours.

TD7 was the chosen algorithm for this task due to its impressive performance over TD3. As stated earlier, its 276.7% performance gain at 300k steps against TD3 on other gym environments, makes it an ideal algorithm for our easy and hardcore versions. TD7 incorporates 4 new features to improve sample efficiency and performance.

State-Action Learned Embeddings (SALE): Learns compact, joint representations of state-action pairs to better model environment dynamics and improve generalisation with input clipping to a fixed interval to mitigate extrapolation error.

Checkpoints: Periodically saves and evaluates the current best policy during training. This is matched with batch training so that training is done using the best policy at the time. This feature does not necessarily improve performance, as in the TD7 study, performance

improvement depended on the environment. This feature was not included due to poor results.

Prioritised Experience Replay (PER): Samples transitions with high temporal difference error more frequently, focusing on the most information-rich experiences [12].

Behaviour Cloning (Offline): Adds a loss term to keep the policy close to the behaviour policy in an offline setting. However, as we are using an online setup this feature was not used [13].

4 CONVERGENCE RESULTS

The model successfully converges and completes both the easy and hardcore versions, shown in Figure 1 and 2. All experiments were conducted using consistent hardware and software configurations on the same machine using seed 42. Specifically, the same versions of Python, PyTorch, NumPy, and Gymnasium were used across all runs to ensure deterministic behavior and reproducibility.

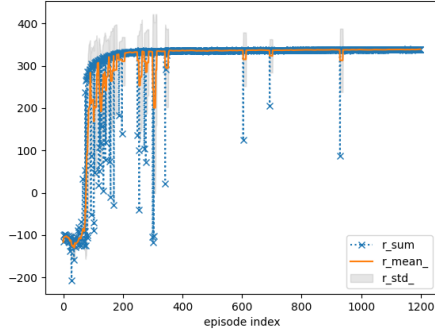


Figure 1: Easy Version

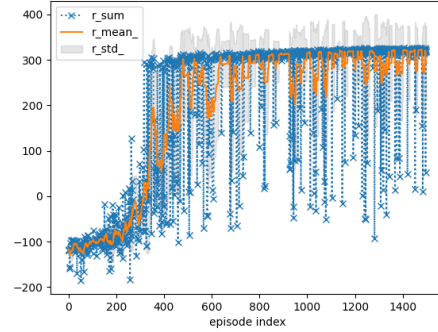


Figure 2: Hardcore Version

The criteria for solving the easy version is to reach 300 reward within 1600 time steps. Our model solves the easy version by episode 95 with 1184 time steps, achieving a reward of 305.84. The model shows signs of convergence from episode 120 onwards. Occasional drops in reward are expected as the model is still learning and failing at edge cases, however as you can see after episode 360, the convergence is strong. A max reward of 339.7 is achieved on episode 1154. The agent successfully learns how to run with optimal movements.

The criteria for solving the hardcore version is to reach 300 reward within 2000 time steps. Our model solves the hardcore version by episode 346 with 1188 time steps, achieving a reward of 306.12. The model shows weak signs of convergence at roughly 700 episodes onwards. Due to the nature of the hardcore version, sharp drops in reward are expected even after 800 episodes. However, in Figure 2, you can see the mean reward is stabilising between 200-300 reward. A max reward of 324.38 was recorded on episode 1137. The agent successfully learns how to walk and overcome the obstacles to solve the hardcore version.

5 EXPERIMENTS

Optuna [14] was used for hyperparameter tuning, which resulted in a significant improvement in the easy version, shown in Figure 3 and 4. However, due to time limitations, the hardcore version was unable to be improved as only 8 trials were run with none beating the original hyperparameters. Within optuna, i needed a value to maximize, and I experimented with a rolling ten-episode average reward and the highest ten-reward average. I achieved better results with the highest ten-reward average.

Further experiments were carried out on the tuned version to implement a decaying exploration noise value as the episode count grew, shown in Figure 5. This wasn't successful, however, switching exploration off after episode 180 resulted in a better convergence after episode 350, as seen in Figure 6. All experiments were done on a CPU and with seed 42 for reproducibility.

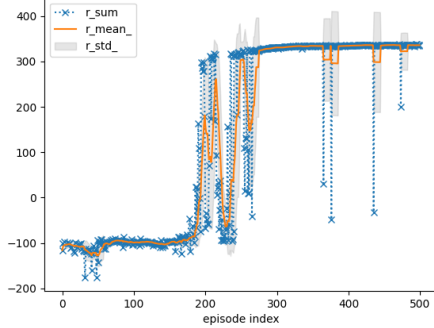


Figure 3: Before Tuning (Easy)

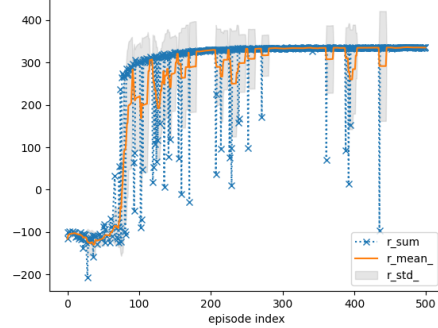


Figure 4: After Tuning (Easy)

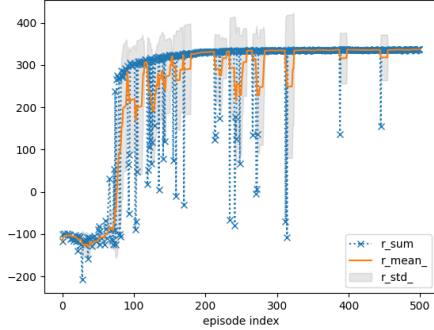


Figure 5: Decay explore noise

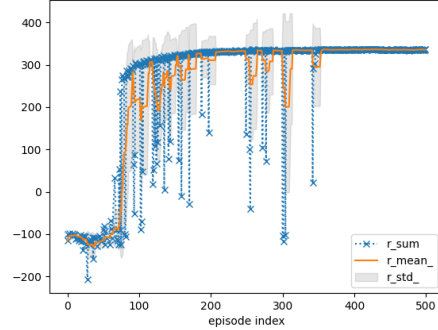


Figure 6: Exploration off after episode 180

6 LIMITATIONS

The model is limited by its time to train, as it takes a considerably longer time to train than TD3. However, the longer training time results in significantly better results. It's also limited in that checkpointing wasn't implemented. From debugging, I believe this is due to the fixed 2000 timesteps we are given and that works against checkpointing. However, as the TD7 paper explored, it could just be not suitable for the bipedal walker used in this project.

FUTURE WORK

In the future, it would be good to implement a more advanced exploration noise reduction algorithm to improve convergence. TD7 could be extended with imagination rollouts taken from the Dreamwalker model to improve sample efficiency and reduce reliance on real environment steps. However, this would drastically increase training time and may be unfeasible.

REFERENCES

- [1] Yan Duan et al. “Benchmarking Deep Reinforcement Learning for Continuous Control”. In: *International Conference on Machine Learning*. PMLR. 2016, pp. 1329–1338. DOI: 10.48550/arXiv.1604.06778. URL: <https://arxiv.org/abs/1604.06778>.
- [2] Scott Fujimoto et al. “For SALE: State-Action Representation Learning for Deep Reinforcement Learning”. In: *arXiv preprint arXiv:2306.02451* (2023). DOI: 10.48550/arXiv.2306.02451. URL: <https://arxiv.org/abs/2306.02451>.
- [3] Scott Fujimoto, Herke van Hoof, and David Meger. “Addressing Function Approximation Error in Actor-Critic Methods”. In: *arXiv preprint arXiv:1802.09477* (2018). DOI: 10.48550/arXiv.1802.09477. URL: <https://arxiv.org/abs/1802.09477>.
- [4] H. Tan. “Reinforcement Learning with Deep Deterministic Policy Gradient”. In: *2021 International Conference on Artificial Intelligence, Big Data and Algorithms (CAIBDA)*. IEEE. 2021, pp. 82–85. DOI: 10.1109/CAIBDA53561.2021.00025. URL: <https://doi.org/10.1109/CAIBDA53561.2021.00025>.
- [5] Arijus Lengvenis. *ArijusLengvenis/bipedal-walker-dreamer*. Jupyter Notebook, GitHub repository. Accessed: 2025-04-29. June 2023. URL: <https://github.com/ArijusLengvenis/bipedal-walker-dreamer>.
- [6] Arsenii Kuznetsov et al. “Controlling Overestimation Bias with Truncated Mixture of Continuous Distributional Quantile Critics”. In: *arXiv preprint arXiv:2005.04269* (May 2020). DOI: 10.48550/arXiv.2005.04269. URL: <https://arxiv.org/abs/2005.04269>.
- [7] Sahil Sinha et al. “D2RL: Deep Dense Architectures in Reinforcement Learning”. In: *arXiv preprint arXiv:2010.09163* (Nov. 2020). DOI: 10.48550/arXiv.2010.09163. URL: <https://arxiv.org/abs/2010.09163>.
- [8] Chenyu Wang and Kris Ross. “Boosting Soft Actor-Critic: Emphasizing Recent Experience without Forgetting the Past”. In: *arXiv preprint arXiv:1906.04009* (June 2019). DOI: 10.48550/arXiv.1906.04009. URL: <https://arxiv.org/abs/1906.04009>.
- [9] Cheng Chen et al. “TransDreamer: Reinforcement Learning with Transformer World Models”. In: *arXiv preprint arXiv:2202.09481* (Nov. 2024). DOI: 10.48550/arXiv.2202.09481. URL: <https://arxiv.org/abs/2202.09481>.
- [10] Kei Ota et al. “Can Increasing Input Dimensionality Improve Deep Reinforcement Learning?” In: *arXiv preprint arXiv:2003.01629* (June 2020). DOI: 10.48550/arXiv.2003.01629. URL: <https://arxiv.org/abs/2003.01629>.
- [11] Greg Brockman et al. “OpenAI Gym”. In: *arXiv preprint arXiv:1606.01540* (June 2016). DOI: 10.48550/arXiv.1606.01540. URL: <https://arxiv.org/abs/1606.01540>.
- [12] Scott Fujimoto, David Meger, and Doina Precup. “An Equivalence between Loss Functions and Non-Uniform Sampling in Experience Replay”. In: *arXiv preprint arXiv:2007.06049* (Oct. 2020). DOI: 10.48550/arXiv.2007.06049. URL: <https://arxiv.org/abs/2007.06049>.
- [13] Scott Fujimoto and Shixiang Shane Gu. “A Minimalist Approach to Offline Reinforcement Learning”. In: *arXiv preprint arXiv:2106.06860* (Dec. 2021). DOI: 10.48550/arXiv.2106.06860. URL: <https://arxiv.org/abs/2106.06860>.
- [14] Takuya Akiba et al. “Optuna: A Next-generation Hyperparameter Optimization Framework”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM. 2019, pp. 2623–2631. DOI: 10.1145/3292500.3330701. URL: <https://doi.org/10.1145/3292500.3330701>.